

生成 AI 利用申告書

- 氏名：塩澤 拓海
- 学籍番号：[TODO: 学籍番号を記入]
- プログラムファイル名：HCK02_03.ino / HCK02_03.pde
- 使用した AI ツール：Claude Code (Opus 4.7)

1. 何に使ったか

発展課題（マイク入力音信号を Processing でスペクトル解析し、結果をウィンドウに描画するスペクトルアナライザを作る）の設計と実装で生成 AI を利用した。提出課題 2 (HCK02_02.ino / HCK02_02.pde) の構成を土台に、Processing 側を時間領域表示から周波数領域 (FFT) 解析へ拡張する作業の補助として用いた。

- Arduino 側 (HCK02_03.ino) のサンプリング周期と Processing 側の SAMPLE_RATE_HZ を一致させるための数値設計 (SAMPLE_US = 150 μ s $\rightarrow$ 6666.67 Hz、ナイキスト約 3333 Hz)
- Processing 側 (HCK02_03.pde) で minim の ddf.minim.analysis.FFT を使ってシリアル受信サンプルを FFT にかけて、ビン強度を dB 表示する設計の相談
- 時間領域波形と周波数領域スペクトルを 1 ウィンドウに上下 2 段で並べるレイアウトと、横軸 Hz 目盛り・縦軸 dB 目盛りの描画方法のレビュー

2. AI への指示（プロンプト）

AI へ送った指示のうち、主要なものを以下に示す。

発展課題は「Processing でスペクトルアナライザを作り、マイクから入力された音信号の波形をスペクトル解析した結果をウィンドウに描画する」プログラムを作りたい。
ベースにしたいファイル：
- Arduino: work/shiozawa/work-0422/HCK02_02/HCK02_02.ino (動作確認済み、6.67 kHz サンプリング)
- Processing: work/shiozawa/work-0422/HCK02_02_processing/HCK02_02.pde (楽曲再生+時間波形描画)

要件：
- Arduino 側はそのまま流用してよい (1 行 1 サンプル送信、150 μ s 周期)
- Processing 側でシリアル受信したサンプルを蓄積し、minim の FFT クラスで解析する
- スペクトルはバーグラフ表示。横軸は Hz、縦軸は dB
- 楽曲再生のキーバインド ('p' / '1'-'6') はそのまま残す

FFT のサイズと窓関数、表示する周波数範囲を決めたい。
- サンプリングレート 6666.67 Hz、ナイキストは約 3333 Hz
- 周波数分解能と応答速度のバランスを考えて FFT_SIZE を決めたい
- 振幅の dB 変換時に $\log_{10}(0)$ で発散しないようにしたい

3. AI の出力の使い方

- ☐ そのまま使った
- ☒ 一部修正して使った
- ☐ 参考にしただけ

→ 上で選んだ理由：

AI が挙げた設計のうち、以下は妥当と判断して採用した：

- Arduino 側は HCK02_02.ino と同じく 1 行 1 サンプル送信を維持し、Processing 側の FFT 用バッファに直接流し込む構成
- `FFT.SIZE = 1024` (2 のべき乗)。サンプリング 6666.67 Hz では 1 回の FFT に必要な時間が約 154 ms で、約 6.5 Hz のビン幅となり、スペアナの応答性と分解能のバランスがとれる
- Hamming 窓を適用してスペクトル漏れを抑える (`fft.window(FFT.HAMMING)`)
- `20 * log10(max(magnitude, 1e-6))` の形で dB 変換し、ゼロ入力時の $-\infty$ 発散を防ぐ
- 上段 (時間領域) / 下段 (周波数領域) の 2 段レイアウト、下段は横軸 Hz・縦軸 dB の目盛りを併記

一方で、AI から追加提案された「FFT 結果を対数周波数軸で表示する」案や「ピークホールドの減衰アニメーション」は、初版では機能を最小に絞ったため今回は採用を見送り、後から拡張する余地として残した。

4. 正しいと確認した方法

[TODO: 実機テスト後に具体的な観測内容を記入する]

想定している確認手順：

- Arduino IDE で HCK02_03.ino がコンパイルでき、シリアルモニタ (921600 bps) に数値が 1 行ずつ流れることを確認する
- Processing IDE で HCK02_03.pde を起動し、上段に時間波形、下段にスペクトルバーグラフが描画されることを観察する
- PC スピーカから 'p' で楽曲を再生し、マイクへ入力したとき、スペクトルが楽曲の音高 (C4 $\approx$ 262 Hz, G4 $\approx$ 392 Hz, A4 = 440 Hz) 付近で立ち上がることを目視で確認する
- '1' (正弦波) で再生したときに、基音 1 本が突出した綺麗なスパイク状スペクトルになることを確認する → 倍音の少ない音色と FFT 結果が直感的に一致するかを見る
- '3' (のこぎり波) や '4' (矩形波) に切り替えると、基音に加えて倍音が高い周波数まで段々と並ぶことを確認する → 倍音構成の理論と実測の対応を確認する
- 拍手や口笛を入力し、無音時とのスペクトル差が見えることを確認する
- `SAMPLE_RATE_HZ` (Processing 側) と `SAMPLE_US` (Arduino 側) の対応が `1000000.0 / SAMPLE_US` で正しく一致していることをソース上で計算しなおして確認する

5. 問題や間違い

- ☐ あった
- ☐ なかった

[TODO: 実機テスト後に、実際に気づいた問題や AI の誤り・自分が気づいた点を記入する]

検証予定ポイント：

- シリアル受信が間に合わずに FFT バッファに穴が空くと、スペクトルが乱れる。`frameRate` と `port.available()` を表示し、取りこぼしが起きていないか確認する
- Arduino のサンプリングジッタが大きいと FFT の周波数軸が歪む。既知の音叉や PC で生成した既知周波数の正弦波 (例: 440 Hz) を入れ、スペクトルのピーク位置が想定ビン (`440 / 6.51  $\approx$  67` 番目) に来るか確認する

- 起動直後に `fftInput` がゼロ初期化のまま FFT にかかるため、最初の数フレームは底値スペクトルが見える。想定どおり徐々にサンプルが埋まるかを観察する
- Processing のシリアルポート名 (`/dev/cu.usbmodem...`) は PC ごとに違うため、メンバ B の環境で書き換え忘れによる起動失敗が起きないか

6. 自分で工夫した点

- Arduino 側 (`HCK02_03.ino`) は `HCK02_02.ino` を継承してサンプリング周期 $150\mu\text{s}$ を維持し、Processing 側の `SAMPLE_RATE_HZ` を `1000000.0 / SAMPLE_US` で導出する形にして、両者の数値がずれない構造にした
- Processing 側 (`HCK02_03.pde`) では、ADC 値をシリアル受信したその場で `[-1, +1]` に正規化して直流成分を抜き、FFT 入力バッファ `fftInput` と時間波形バッファ `waveBuf` の両方に書き込む構成にした → 1 サンプルで時間領域・周波数領域の両方を更新できる
- FFT の縦軸を $20 * \log_{10}(\max(\text{magnitude}, 1e-6))$ で dB 化し、`SPECTRUM_DB_MIN / SPECTRUM_DB_MAX` の範囲外を `constrain` でクランプすることで、ゼロ入力時にバーが画面下に飛ぶ／対数の発散で描画が壊れる、といった事故を防いだ
- 表示する最高周波数 `DISPLAY_MAX_HZ` を 3000 Hz に絞り、ナイキスト直近の使い物にならない領域を捨てて可聴域（楽器音域）に画面を集中させた
- バーの色を周波数比 `i / numBins` で連続的に変化させ、どの帯域が立っているかを色でも認識しやすくした（初期方針の「落ち着いた配色」の範囲内で、彩度を控えめに調整）
- 楽曲再生のキーバインド (`'p' / '1'-'6'`) を `HCK02_02.pde` から維持し、波形 → 周波数の対応を音色を切り替えながら直感的に検証できるようにした

参考文献

- 生成 AI 利用申告書テンプレート（ハッカソン 1 講義資料, [references/lectures/](#)）
- 提出課題 2 のコード一式（本リポジトリ `work/shiozawa/work-0422/HCK02_02/` 配下）
- 動作確認済み Arduino: `work/shiozawa/work-0422/HCK02_02/HCK02_02.ino`
- Processing Reference: `Serial`, `bufferUntil`, `serialEvent` <https://processing.org/reference/libraries/serial/>
- Minim Reference: `FFT`, `AudioOutput`, `Oscil`, `Line` <https://code.compartmental.net/minim/>
- Minim FFT Javadoc: <https://code.compartmental.net/minim/javadoc/ddf/minim/analysis/FFT.html>